



Structured Query Language SQL

28 Tech

Các câu lệnh cơ bản

Data Types - Kiểu dữ liệu

So nguyên (Integer Types):

- **TINYINT** (1 byte): Lưu giá trị từ -128 đến 127 (hoặc 0-255 nếu **UNSIGNED**).
- **SMALLINT** (2 bytes): -32,768 đến 32,767.
- **MEDIUMINT** (3 bytes): -8,388,608 đến 8,388,607.
- **INT** (4 bytes): -2,147,483,648 đến 2,147,483,647.
- **BIGINT** (8 bytes): -9,223,372,036,854,775,808 đến 9,223,372,036,854,775,807.

quantity **SMALLINT**



Data Types - Kiểu dữ liệu

Số thực (Floating-Point Types):

- `FLOAT(M,D)`: Giá trị dấu phẩy động chính xác thập.
- `DOUBLE(M,D)`: Giá trị dấu phẩy động chính xác cao hơn.
- `DECIMAL(M,D)`: Giá trị số chính xác cao, phù hợp với lưu trữ tiền tệ.

```
amount DECIMAL(10,2)
```

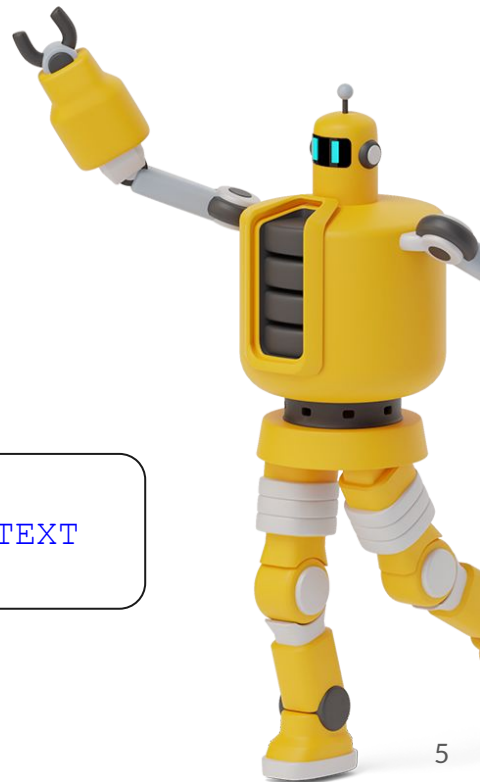


Data Types - Kiểu dữ liệu

Kiểu chuỗi (String Types)

- **CHAR(N)**: Lưu chuỗi ký tự có độ dài cố định (tối đa 255 ký tự).
- **VARCHAR(N)**: Lưu chuỗi ký tự có độ dài thay đổi (tối đa 65,535 ký tự).
- **TEXT**: Lưu trữ văn bản dài (tối đa 4GB)

```
username VARCHAR(50) about TEXT
```



Data Types - Kiểu dữ liệu



Kiểu ngày và thời gian (Date and Time Types)

- **DATE**: Lưu ngày theo định dạng **YYYY-MM-DD**.
- **DATETIME**: Lưu cả ngày và giờ theo định dạng **YYYY-MM-DD HH:MM:SS**.
- **TIMESTAMP**: Giống **DATETIME** nhưng được cập nhật tự động khi có thay đổi dữ liệu.

```
order_date DATETIME
```



So sanh trong MySQL

Cac toan tu so sanh: =, !=, <>, >, <, >=, <=

So sanh chuoi, so, ngay thang.

```
age >= 18
```



Ràng buộc toàn vẹn trong bảng

- Các loại ràng buộc nhất quán
 - primary key (A1, ..., An)
 - foreign key (Am, ..., An) references r
 - not null
- SQL ngăn chặn mọi cập nhật vào CSDL vi phạm ràng buộc toàn vẹn.

```
create table instructor (  
    ID                char(5) ,  
    name              varchar(20) not null ,  
    dept_name          varchar(20) ,  
    salary             numeric(8,2) ,  
    primary key (ID) ,  
    foreign key (dept_name) references department);
```



Vi du them .

- **create table** *student* (
 ID **varchar**(5),
 name **varchar**(20) not null,
 dept_name **varchar**(20),
 tot_cred **numeric**(3,0),
 primary key (*ID*),
 foreign key (*dept_name*) **references** *department*);

- **create table** *takes* (
 ID **varchar**(5),
 course_id **varchar**(8),
 sec_id **varchar**(8),
 semester **varchar**(6),
 year **numeric**(4,0),
 grade **varchar**(2),
 primary key (*ID*, *course_id*, *sec_id*, *semester*, *year*) ,
 foreign key (*ID*) **references** *student*,
 foreign key (*course_id*, *sec_id*, *semester*, *year*) **references** *section*);

Vi du them .



```
■ create table course (  
    course_id    varchar(8),  
    title        varchar(50),  
    dept_name    varchar(20),  
    credits      numeric(2,0),  
    primary key (course_id),  
    foreign key (dept_name) references department);
```

Thay đổi Cấu trúc Bảng

- ❖ Drop Table r
- ❖ Alter

alter table r add A D

- Trong đó, A là thuộc tính (cột) mới được thêm vào bảng 'r', D là kiểu dữ liệu của A
- Tất cả các bản ghi trước đây sẽ được gán **null** cho cột mới này

alter table r drop A

- Trong đó, A là thuộc tính (cột) muốn xóa

Cấu trúc cơ bản của truy vấn

❖ Cấu trúc của 1 SQL Query:

```
select  $A_1, A_2, \dots, A_n$   
from  $r_1, r_2, \dots, r_m$   
where  $P$ 
```

- ❑ A_i là danh sách các thuộc tính
- ❑ R_i là bảng
- ❑ P là điều kiện lọc.

❖ Kết quả của câu truy vấn là **1 quan hệ**

Menh đề **Select**

- ❖ **select** liệt kê các thuộc tính mong muốn trong kết quả của truy vấn
- ❖ Tìm **tên** của tất cả các **giảng viên**

```
select distinct dept_name  
from instructor
```

- ❖ Name trong SQL là Không phân biệt HOA-thường (Name $\equiv$ NAME $\equiv$ name)

Menh đề **Select**

- ❖ SQL cho phép trùng lặp trong các quan hệ cũng như trong kết quả truy vấn. SQL cho phép trùng lặp trong các quan hệ cũng như trong kết quả truy vấn.

- ❖ Để loại bỏ các mục trùng lặp, hãy chèn từ khóa **distinct** sau lệnh select.

```
select distinct dept_name  
from instructor
```

dept_name

Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.

Menh đề [^]Select[^]

- ❖ Dấu ^{*} thể hiện muốn **lấy tất cả** các trường

```
select *  
from instructor
```

- ❖ Một thuộc tính có thể lấy mà không cần **from**

- Kết quả là 1 bảng có 1 cột và 1 hàng có giá trị là '467'
- Có thể đặt tên cho cột

```
select '437'
```

```
select '437' as FOO
```

Menh đề **Select**

- ❖ Mệnh đề **select** có thể chứa các biểu thức **số học** liên quan đến phép toán +, −, *, /, và phép toán trên hằng số hoặc thuộc tính.

```
select id, supplierId * unitPrice
from Product
```

```
select id, supplierId * 10
from Product
```

- Trả về 1 bảng có cấu trúc giống bảng 'Product' ngoại trừ cột `supplierId * 10`

```
select ID, name, salary/12
from instructor
```

- Sửa tên cột trả về

```
select ID, name, salary/12 as monthly_salary
```


Mệnh đề **From**

- ❖ Mệnh đề **from** chỉ định tất cả các quan hệ liên quan tới câu query
- ❖ VD: Để tìm tất cả các trường hợp có thể xảy ra của cặp `instructor`, `teaches`

```
select *  
from instructor, teaches
```

- Những thuộc tính trùng tên (VD: ID) sẽ được đổi tên thành `instructor.ID`
- ❖ Câu lệnh trên, có thể tạo ra những trường hợp '**không khớp nhau**'
- ❖ Thông thường, không sử dụng một cách trực tiếp như trên. Nên kết hợp với `where`

Ví dụ From .

- ❖ Tìm tên của tất cả giảng viên đã **tham gia dạy ít nhất 1 lớp học**, trả về cả course_id

```
select name, course_id
from instructor , teaches
where instructor.id = teaches.instructor_id
```

- ❖ Tìm tên của tất cả giảng viên **thuộc khoa CNTT1** đã tham gia dạy ít nhất 1 lớp học, trả về cả course_id

```
select name, course_id
from instructor , teaches
where instructor.id = teaches.instructor_id
      and instructor.dept_name = 'CNTT1'
```

Toán tử Đổi tên (Rename)

- ❖ SQL cho phép đổi tên các **quan hệ** và **thuộc tính** bằng cách sử dụng **AS**

```
select name, course_id
from instructor as I, teaches as TE
where I.id = TE.instructor_id
```

- ❖ Liệt kê các cặp sinh viên cùng thành phố

```
select A.student_code, B.student_code
from students as A, students as B
where A.student_code < B.student_code
and A.city = B.city
```

- ❖ Từ khóa 'AS' là không bắt buộc $\Rightarrow$ instructor **as** T $\equiv$ instructor T

INSERT INTO



- ❖ Câu lệnh này **INSERT INTO** được sử dụng để chèn bản ghi mới vào bảng.
- ❖ Chỉ định cả tên cột và giá trị cần chèn:

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

- ❖ Nếu muốn thêm giá trị cho tất cả các cột của bảng, cần đảm bảo đúng thứ tự của các cột trong bảng

```
INSERT INTO table_name  
VALUES (value1, value2, value3, ...);
```

INSERT INTO

```
INSERT INTO instructor (full_name, dept_name, salary) VALUES
('Nguyễn Văn An', 'Khoa Công nghệ Thông tin', 25000000),
('Trần Thị Bình', 'Khoa Điện tử Viễn thông', 28000000),
('Lê Minh Cường', 'Khoa Công nghệ Thông tin', 30000000),
('Phạm Thanh Dung', 'Khoa Khoa học Máy tính', 27000000),
('Hoàng Ngọc Em', 'Khoa Kỹ thuật Máy tính', 26000000),
('Vũ Thị Phương', 'Khoa Điện tử Viễn thông', 29000000),
('Đỗ Minh Quân', 'Khoa Khoa học Dữ liệu', 32000000),
('Ngô Lan Hương', 'Khoa Khoa học Máy tính', 27500000),
('Bùi Quang Tuấn', 'Khoa Kỹ thuật Máy tính', 26800000),
('Trịnh Mai Linh', 'Khoa Công nghệ Thông tin', 31000000),
('Lý Thanh Tùng', 'Khoa Khoa học Dữ liệu', 33000000),
('Mai Thị Hoa', 'Khoa An toàn Thông tin', 28500000),
('Đinh Quốc Việt', 'Khoa An toàn Thông tin', 29500000);
```